Software Uninstaller — Pro

User and administrator manual | Version 1.3.1

Release documentation | October 2026

Use only on systems you own or are authorized to administer. Review all commands before running them.

1. Overview

AdminOps Software Uninstaller Pro v1.3.1 inventories registered Windows software and provides a GUI and enterprise command-line interface for removal.

It supports registered MSI uninstall commands and vendor-registered quiet uninstall commands. Interactive EXE uninstall is available only with explicit approval or the CLI opt-in switch. The tool does **not** guess undocumented silent switches.

2. Requirements and installation

Windows 10/11 or Windows Server with Windows PowerShell 5.1. The GUI needs an interactive desktop; it is not suitable for Server Core. Machine-wide uninstalls generally require administrator rights. No external PowerShell modules are required.

Sign in to the Member Dashboard with an active Pro membership and download **AdminOps-Software-Uninstaller-Pro-v1.3.1.zip**.

Extract the complete archive. Keep **Pro-Edition** and its **Core**, **GUI**, and **Enterprise** subfolders together. For uninstall operations, open PowerShell with appropriate privileges.

3. Using the graphical interface

From **Pro-Edition**, double-click **Start-GUI.cmd**. Alternatively run:

```
powershell.exe -NoProfile -STA -File .\GUI\AdminOps-Uninstaller-GUI.ps1
```

Review the installed software list, search by name, select the intended application, and choose **Uninstall Selected**. Read and confirm each prompt before proceeding.

If an EXE only has a registered interactive uninstall command, the GUI offers an additional confirmation. The vendor wizard may open and require input. Cancel if the application or command is not what you expected.

The GUI may appear unresponsive while a vendor uninstaller runs. Check the vendor wizard, Task Manager, and log before launching another removal. Refresh the inventory to verify the entry disappears.

4. Enterprise CLI: inspect and preview

Open an elevated PowerShell window in the **Pro-Edition** folder. Start with read-only listing and a WhatIf preview:

```
.\Enterprise\Uninstall-Software.ps1 -List
.\Enterprise\Uninstall-Software.ps1 -Name "Example App" -Exact -WhatIf
```

Replace **Example App** with the exact registered display name. **-WhatIf** previews the intended action without starting the uninstaller.

5. Enterprise CLI: remove software

Only after verifying the target, run a removal:

```
.\Enterprise\Uninstall-Software.ps1 -Name "Example App" -Exact -Force -Confirm:$false
```

To explicitly remove every matching entry (potentially destructive):

```
.\Enterprise\Uninstall-Software.ps1 -Name "Example" -AllMatches -Force -Confirm:$false
```

To permit a vendor-registered interactive EXE uninstaller when no quiet command is available:

```
.\Enterprise\Uninstall-Software.ps1 -Name "Example App" -Exact -AllowInteractiveExe -Force
-Confirm:$false
```

Do not use -AllowInteractiveExe in unattended SCCM/Intune deployments. Interactive vendor installers can wait indefinitely for user input. Without -Exact, a name search can match multiple applications; inspect matches before choosing -AllMatches.

6. Deployment with SCCM / Intune

Package the entire **Pro-Edition** folder structure and set the working directory to its root. Example deployment command:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File ".\Enterprise\Uninstall-Software.ps1" -Name "
Example App" -Exact -Force -Confirm:$false
```

For machine-wide software, deployment may run under SYSTEM, subject to your organization’s policy. Use only applications with a suitable registered unattended uninstall command. Test on a small device group first.

Documented exit codes: **0** success/preview; **1** failure; **2** no match; **3** multiple matches; **4** missing -Force; **3010** reboot requested. The engine treats MSI return code 1641 as a reboot request. Configure your deployment platform accordingly.

7. Logs and validation

Default log: **%ProgramData%\AdminOps\Uninstaller\uninstall.log**. For the CLI, supply **-LogPath** to choose a different path.

After removal, refresh the inventory, inspect the log and verify that the application is actually gone. A vendor interactive wizard may report success even if a user cancels within that wizard.

8. Safety, scope and limitations

Run only on authorized, trusted systems. Registry uninstall commands can execute programs; inspect them and back up critical data before removal.

The tool does not remove Store/UWP packages, device drivers, arbitrary leftover files or registry remnants, or bypass tamper protection. It does not terminate running processes. It avoids Win32_Product enumeration, which can trigger MSI repairs.

Some vendor uninstallers require interaction, have dependencies, or return misleading status codes. Test each vendor and version before fleet deployment. No account login, subscription check, network connection or telemetry is built into the local scripts.

9. Troubleshooting

No matches: confirm the installed display name using -List. **Multiple matches:** use -Exact or review all results before -AllMatches. **Permission errors:** verify elevation and policy.

GUI seems stuck: look for a vendor uninstall dialog behind other windows and inspect the log; avoid repeated clicks. **Still installed:** check exit status, log, and vendor-specific requirements.

Interactive EXE launch failure: v1.3.1 fixes a Start-Process issue when a registered interactive EXE command has no arguments. Verify you downloaded v1.3.1, not v1.3.

10. Release notes and support

Version **1.3.1** includes a launcher hotfix for registered interactive EXE commands with blank argument strings. Version 1.3 added an explicit GUI interactive fallback and improved status messages.

User testing has covered inventory, search, WhatIf, confirmations, cancellation, and actual uninstalls with logging; compatibility is not guaranteed for every vendor or enterprise deployment.

Website: AdminOps Toolkit | Member Dashboard